Especialización QLoRA del agente Redactor en normativa UdeC

Entregable 4 · Fine-tuning supervisado sobre Qwen/Qwen3-4B en arquitectura multi-agente

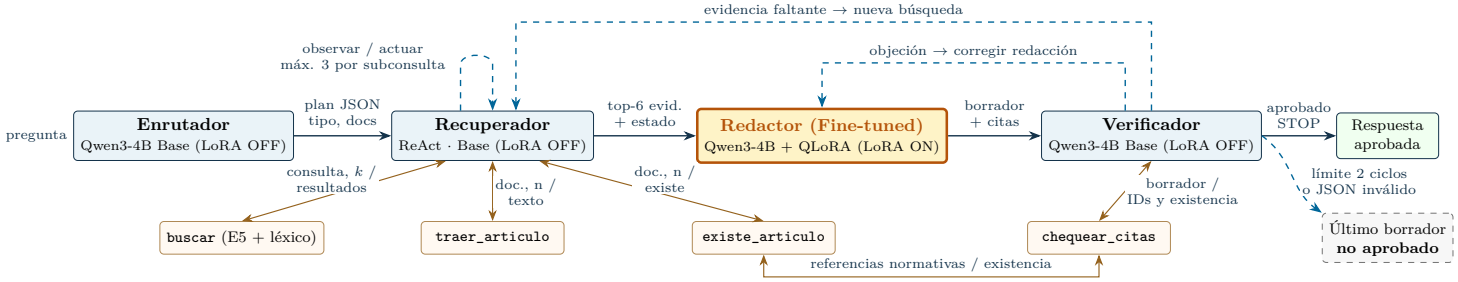
Generative Artificial Intelligence (580694) · Segundo semestre 2026 Álvaro Contreras y Pablo Cortés

Repo: Deliverable 4 (deliverable-4) · **HF (abierto):** Pablox0333/qwen3-4b-redactor-udec-d4 · **Video (≤3 min):** Guía/script en video/ (link en README)

1. Continuidad D1–D3, elección empírica del agente y restricción de modelo

Continuamos D1–D3 sobre normativa UdeC (**197 unidades / 201 fragmentos** de RG, RI-FI y Calendario 2026), embeddings E5, las **75 preguntas oficiales** (50 originales + 25 difíciles) y el grafo de cuatro agentes sobre **Qwen/Qwen3-4B** (NF4, T4, $T = 0{,}7$, modo *thinking* desactivado). Al auditar las trazas v6 de D3 (**66/75**), **8 de los 9 errores finales (88,9 %)** (`P13, P24, P26, P36, H07, H14, H18, H25`) ya contaban con la evidencia necesaria en el `top-6` del **Redactor**, además de 11 fallos en primer borrador (`sin_verificador`: 63/75) y 25 rechazos hacia ese rol. Seguir ajustando prompts fracasó: en D3 v5 editar `03_redactor.txt` bajó la exactitud de 58/75 a 55/75 (obligando a revertir a v4), y desactivar CoT (`sin_cot`: 65/75) casi empató al sistema completo. Por ello fine-tuneamos únicamente al **Redactor** desde `Qwen/Qwen3-4B`; en inferencia, `Enrutador`, `Recuperador` y `Verificador` conservan pesos base vía `disable_adapter()` (565/565 llamadas OFF, 101/101 Redactor ON, **0 violaciones**, máx $|\Delta\text{logits}| = 14{,}06$).

2. Grafo multi-agente con aislamiento LoRA en el Redactor



Solo **Redactor** carga el adaptador LoRA entrenado; los otros tres roles mantienen pesos base `Qwen/Qwen3-4B` y prompts `prompts/01-04` idénticos a D3 v6.

3. Datos de entrenamiento y validación antifuga

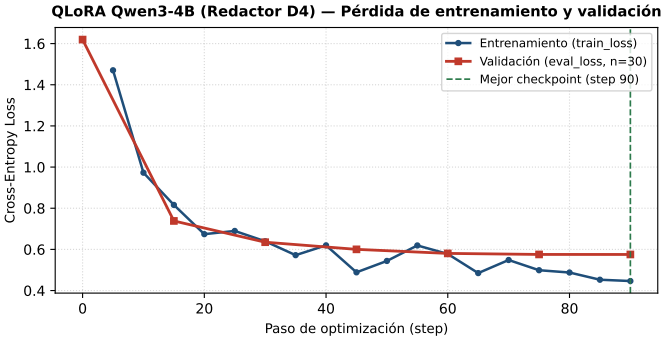
Construimos un corpus supervisado de **210 pares híbridos (180 train / 30 val)** en `redactor_train.jsonl` y `redactor_val.jsonl` bajo el esquema exacto de `03_redactor.txt` (`<pensar>...</pensar>` + `Respuesta / Condiciones o límites / Fuente`). Cada ejemplo ensambla **3–6 unidades literales** de `corpus.json` (mediana 6,0; barajadas, `seed=202604`), cubriendo 163/197 unidades del corpus.

Bloques (train/val): `citas_cruce` (35/5), `condiciones_excepciones` (35/5), `polares_y_lectura` (30/5), `calculos_y_orden` (30/5), `resiliencia_feedback` (25/5) y `abstencion_honesta` (25/5); estados: `suficiente` (192), `insuficiente` (13) y `no_existe` (5).

Autoría, revisión humana y antifuga: Todas las salidas fueron curadas determinísticamente por el equipo en Python sobre artículos literales del reglamento (sin modelo externo generador). Auditamos a mano una muestra estratificada de **30 casos** (`MUESTRA_REVISION_MANUAL.md`: 3 train + 2 val por bloque). Frente a las 75 consultas de evaluación: **0 duplicados, 0 coincidencias exactas y 0 alertas** sobre umbral (seq_ratio $\geq 0{,}85$ o Jaccard $\geq 0{,}80$), con **0 ejemplos truncados** a 4096 tokens ($\bar{L}_{\text{tr}} = 1907{,}4$, $\bar{L}_{\text{val}} = 1993{,}4$).

4. Configuración QLoRA y curva de pérdida

Entrenamiento QLoRA de `Qwen/Qwen3-4B` (4-bit NF4, doble cuantización, `float16`) en **Tesla T4 (15,64 GB VRAM)** durante **54,75 min (3285 s, 90 pasos / 3,0 épocas)**. LoRA $r = 16$, $\alpha = 32$, `dropout`=0,05 en 7 proyecciones (`q,k,v,o,gate,up,down_proj`: 33,03M entrenables = **1,48 %** de 2,24B), `batch`=1, `grad_accum`=6 (efectivo 6), `lr`=10^{-4} coseno (`warmup`=9), `paged_adamw_8bit` y `completion_only_loss=True`. `eval_loss` baja de **1,6195** (paso 0) a **0,5753** (paso 90), y `train_loss` de 1,4703 (paso 5) a 0,4463 (paso 90).



Checkpoint óptimo (paso 90) publicado con acceso abierto en HF: `Pablox0333/qwen3-4b-redactor-udec-d4`.

5. Resultados: cuatro sistemas, control y comparativa

Evaluamos las 75 preguntas con la pauta estricta heredada de D2/D3 (`LEEME_PAUTA.md`, incluyendo Regla 3 / `P01` por afirmaciones extra mal citadas).

Sistema / Control	Orig. (50)	Dif. (25)	Total (75)	Llam./q
Directo D1 (control)	1 (2 %)	0 (0 %)	1 (1,3 %)	1,0
RAG D2 (E5)	40 (80 %)	16 (64 %)	56 (74,7 %)	1,0
Multi-agente D3 Base	**46 (92 %)**	**20 (80 %)**	**66 (88,0 %)**	9,5
Multi-agente D4 (FT)	45 (90 %)	19 (76 %)	64 (85,3 %)	**8,9**
D4 núcleo (sin P01)	*47 (94 %)*	*21 (84 %)*	*68 (90,7 %)*	*8,9*

0 fallbacks y 0 violaciones LoRA. Tiempo T4: 137,6 min (D3) → **114,4 min** (D4, **−16,9 %**); llamadas reales: 711 → **666** (**−6,3 %**); casos >1 ciclo: 21 → **15** (**−28,6 %**).

Mejoras y control pareado (6 casos idénticos): D4 resuelve **6/9 fallos estrictos de D3** (`P13, P24, P26, H07, H14, H25`) y logra 100 % en `numerica` (10/10), `condicional` (10/10), `negacion_excepcion` (3/3), `abstencion` (10/10), `colision_lexica` (5/5) y `premisa_falsa` (4/4). En la comparativa controlada en el mismo entorno (`comparativa_base_vs_finetuned.json`), FT logra **4/6 vs 1/6** del base: resuelve `H07` en 1 ciclo (42/4 = 10,5; Base falla tras 227,7 s) y elimina la contradicción polar de `H14`.

6. Fallos, degradaciones y cuello arquitectónico

El delta estricto respecto a D3 (64 vs 66) expone tres mecanismos empíricos guardados en `evaluacion/casos_fallo_y_regresion.json`:

1) `H03` (fallo persistente Recuperador + sobreconfianza FT): `CAL-2026-S2-02` (10 ago) queda fuera del `top-6`; en lugar de abstenerse como D3, el Redactor FT calcula 04 sep → 11 dic = 98 días (`[CAL-2026-S2-04]`).

2) `H19` y `H06` (degradación por Verificador base): En `H19`, el Redactor FT responde **exactamente bien** según `[RG-ART-028]` en ciclos 1–2, pero el Verificador base objeta pidiendo umbrales de `RI-FI-ART-014` (15/10 créditos) y fuerza un ciclo 3 erróneo. En `H06`, rechaza 38/4 = 9,5 en ciclo 1 y aprueba la retracción en ciclo 2. En total, **8/68 aprobaciones son incorrectas** y **4/7 rechazos eran válidos**.

3) Desvíos de citas `[ID]` y mezcla en Condiciones (`P01`): El Redactor FT sustituye IDs (`P6`: `RG-ART-010` fuera de evidencia; `P36`: `CAL-2026-S2-04`; `H04`: `RG-ART-003-DEF-10`), acota mal `P10` (4 semanas) y añade cláusulas mal citadas en Condiciones (`P7, P31, H02, H18`), situando al `Verificador` y al `top-6` como siguiente cuello de mejora.

7. Reproducibilidad

En `Deliverable 4/`: `scripts/` (`construir_dataset.py`, `entrenar_redactor.py`, `evaluar_d4.py`, `generar_auditoria_d4.py`), prompts versionados `prompts/01-04`, registros en `evaluacion/` (`trazas_D4.json`, `auditoria_D4.json`, `casos_fallo_y_regresion.json`), notebooks Colab y guía/soporte en `video/`.